



DuocUC[®] INFORMÁTICA Y
TELECOMUNICACIONES

■ Redes Fully connected Feed Forward

- TLY1102

A black and white photograph of a man in a suit standing in a modern office, holding a tablet and smiling. The office has glass partitions and modern lighting. In the foreground, there is a desk with a coffee cup and some papers. The overall atmosphere is professional and contemporary.

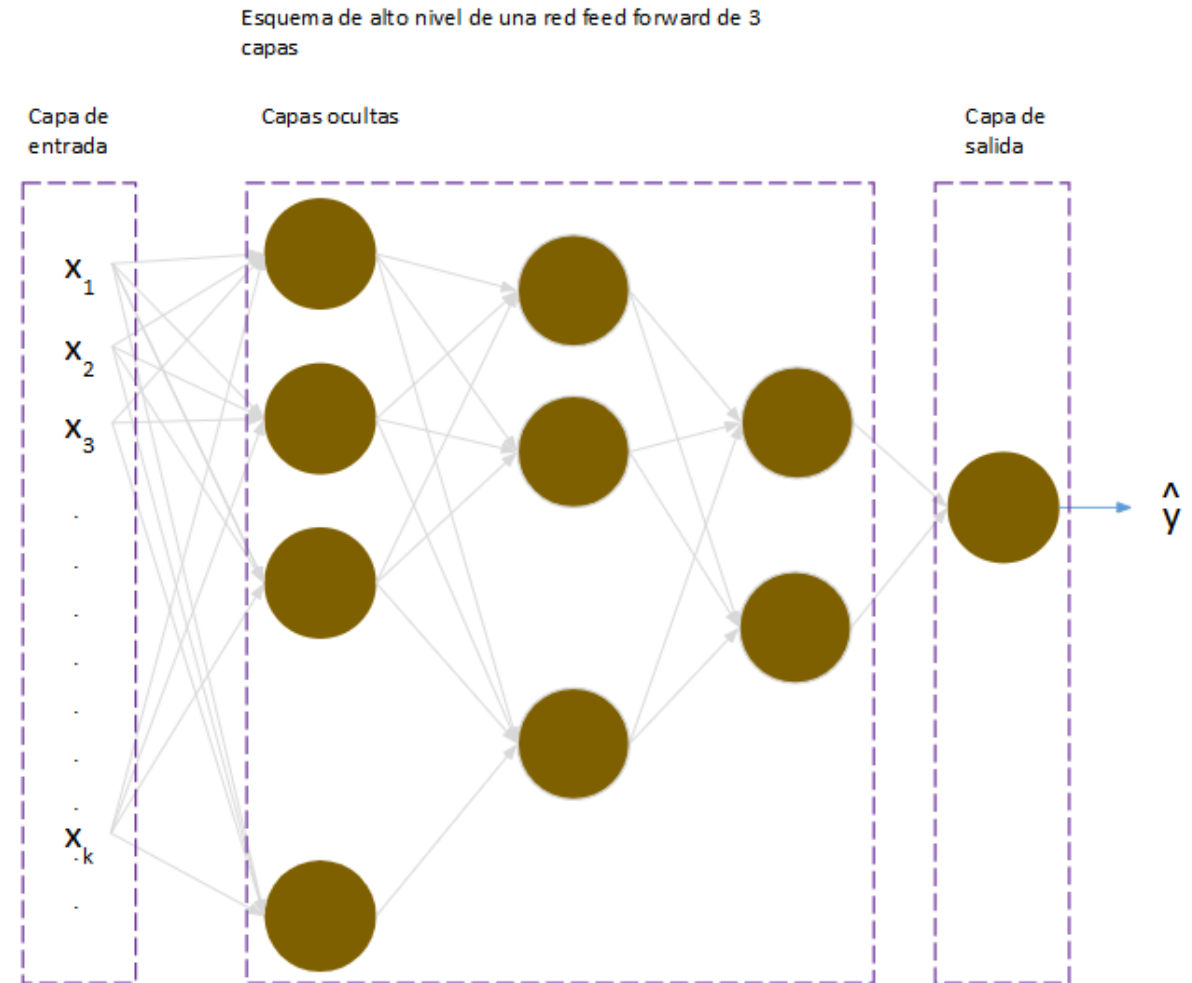
01

Fundamentos

• Redes Feed Forward Fully Connected

Se denominan redes feed forward fully connected al modelo base de redes neuronales, donde **la información fluye hacia adelante y las “neuronas” están inter conectadas todas con todas.**

Desde ahora, ya comenzamos a trabajar con el concepto de redes neuronales profundas (Deep Learning) y el nombre se debe a que **las redes pueden tener muchas capas ocultas**, como lo vemos en esta gráfica.

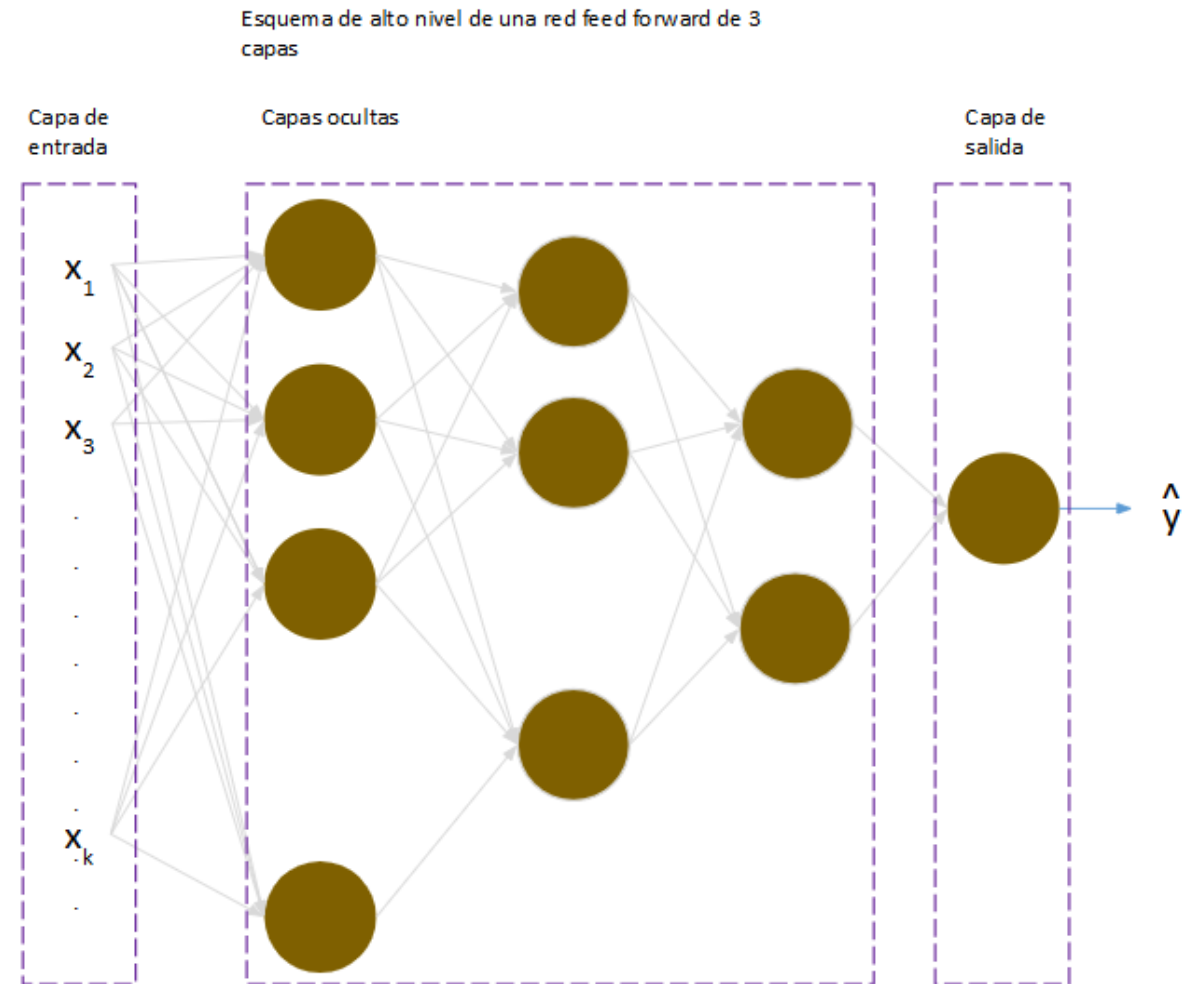


• Redes Feed Forward Fully Connected

Al igual que para determinar **la cantidad de neuronas por capa** no existe una **fórmula**; tampoco existen una para **definir la cantidad de capas** de una red neuronal profunda.

Es importante considerar, que **al aumentar la cantidad de neuronas por capa y/o el número de capas** de la red, **aumentamos drásticamente la cantidad de parámetros** (pesos (w) y bias (b)).

De la misma manera **aumenta el tiempo de duración del “entrenamiento”** de la red.

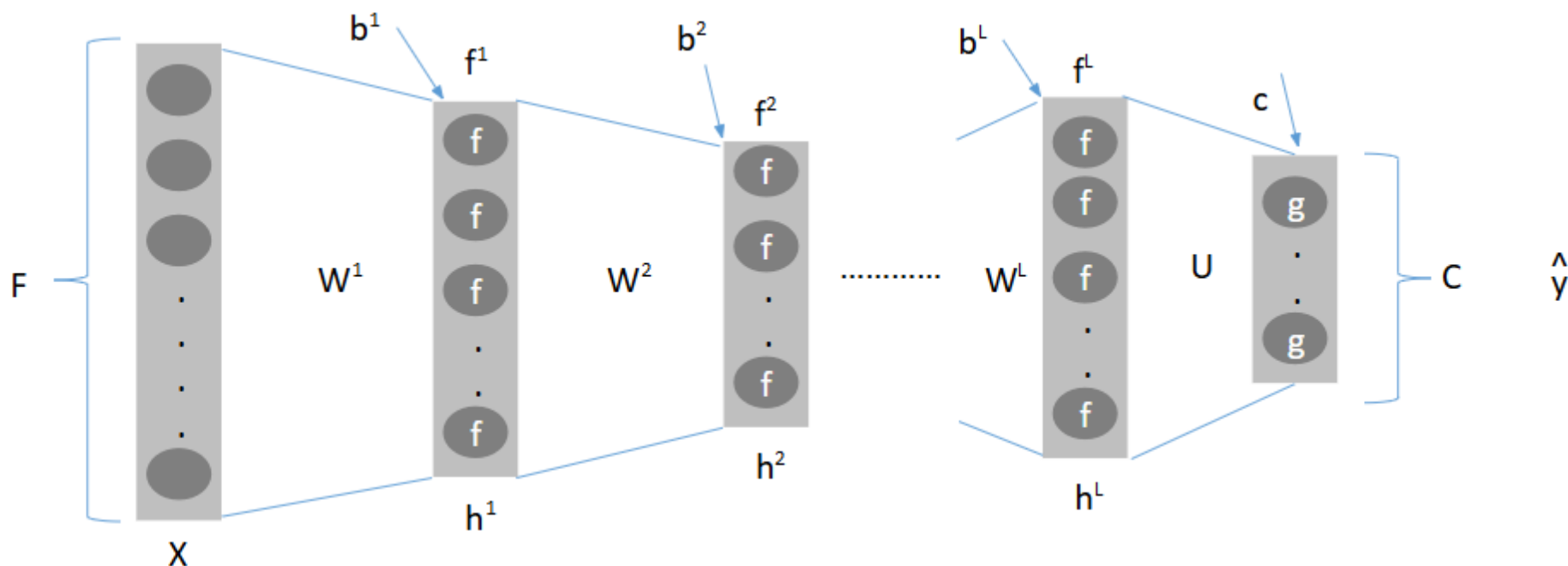


• Redes Feed Forward Fully Connected

Notación matemática

Recordemos que la entrada se denomina capa de entrada y es un vector, **la representaremos una X .**

Los pesos, que también son una matriz, **la denominamos con la letra W mayúscula** y le vamos a poner un superíndice “ i ” que indica el número de la capa a la cual pertenece.

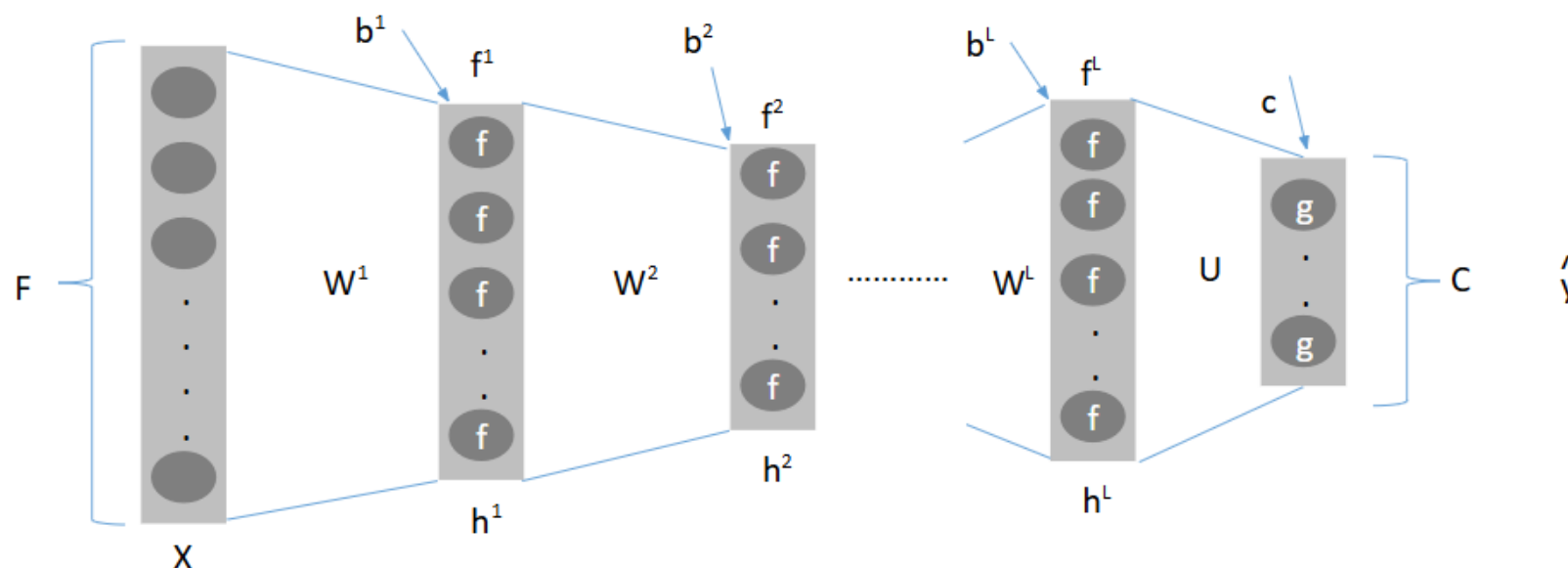


• Redes Feed Forward Fully Connected

Notación matemática

Cada capa tiene una función de activación “ f ” con su respectivo superíndice indicando el número de la capa a la cual se refiere, y **cada capa oculta “ h ”**, su vez tiene un vector de sesgo o bias en jerga de Deep learning, que **representamos con la letra b** y también le agregamos un superíndice para saber a que capa pertenece.

La **cantidad de capas escondidas** la vamos a denotar con la letra “ L ”, hasta llegar a la capa de salida.

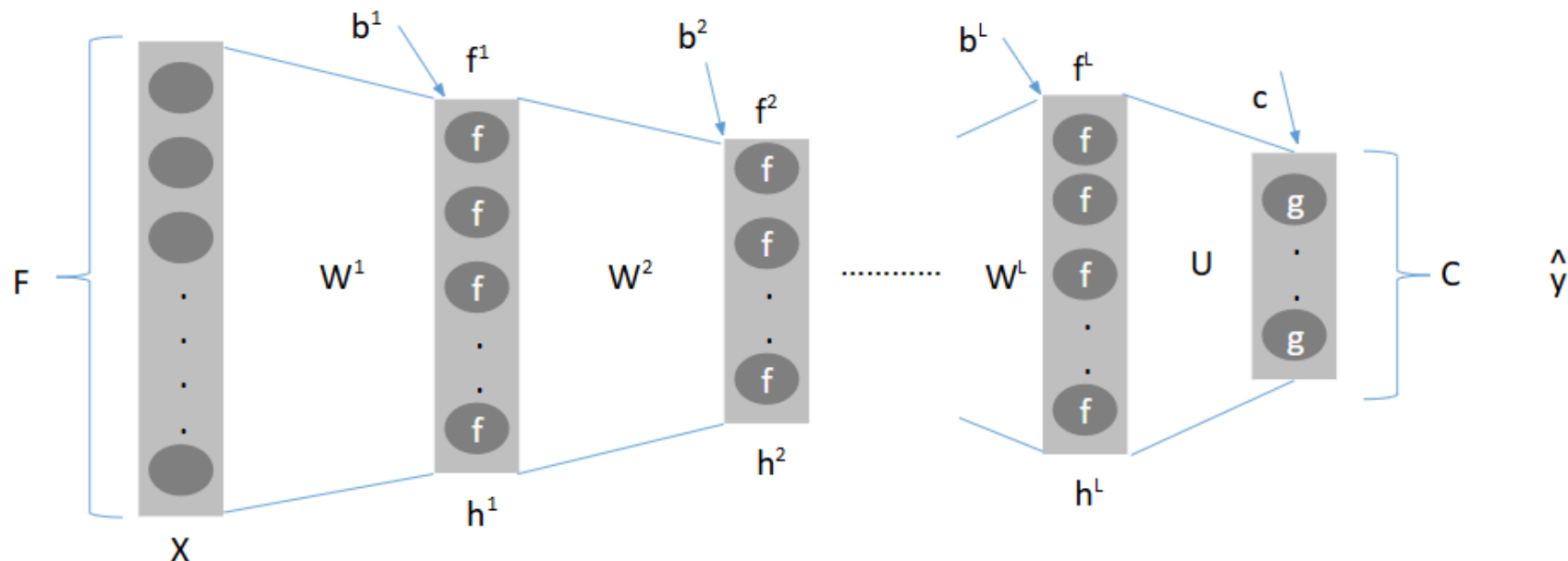


• Redes Feed Forward Fully Connected

Notación matemática

Definiremos con U a la matriz que tiene los parámetros de la última capa, previa a la capa de salida. A su vez, la capa de salida tiene una **función de salida** que la definimos con la letra “ g ” y un **bias** cuya notación es la letra “ c ” minúscula.

El **tamaño de la capa de entrada** la vamos a denotar por la letra “ F ” y el **tamaño de la capa de salida** “ C ” (cantidad de neuronas), que puede ser un vector, es decir una red que arroje como salida varias opciones.



• Redes Feed Forward Fully Connected

Notación matemática

Matemáticamente, la representación gráfica anterior se transforma en lo siguiente:

La salida de la capa oculta h^i es igual a la función de activación de la capa i , aplicada a la salida de la capa anterior h supra i menos 1 multiplicado por la matriz W supra " i " que contiene los pesos de la capa supra " i ", más el vector de bias b de la capa i .

Para que funcione la fórmula, asumimos que h supra 0 es igual a X .

$$h^{(i)} = f^{(i)}(h^{(i-1)} * W^{(i)} + b^{(i)})$$

Finalmente, la representación de la capa de salida como función es: " y " gorro, es igual a la función de salida g , aplicada a la última capa h super índice L , multiplicado por la matriz U (pesos de la última capa) más el vector de sesgo que lo denominamos con la letra c .

$$\hat{y} = g(h^{(L)} * U + c)$$

• Redes Feed Forward Fully Connected

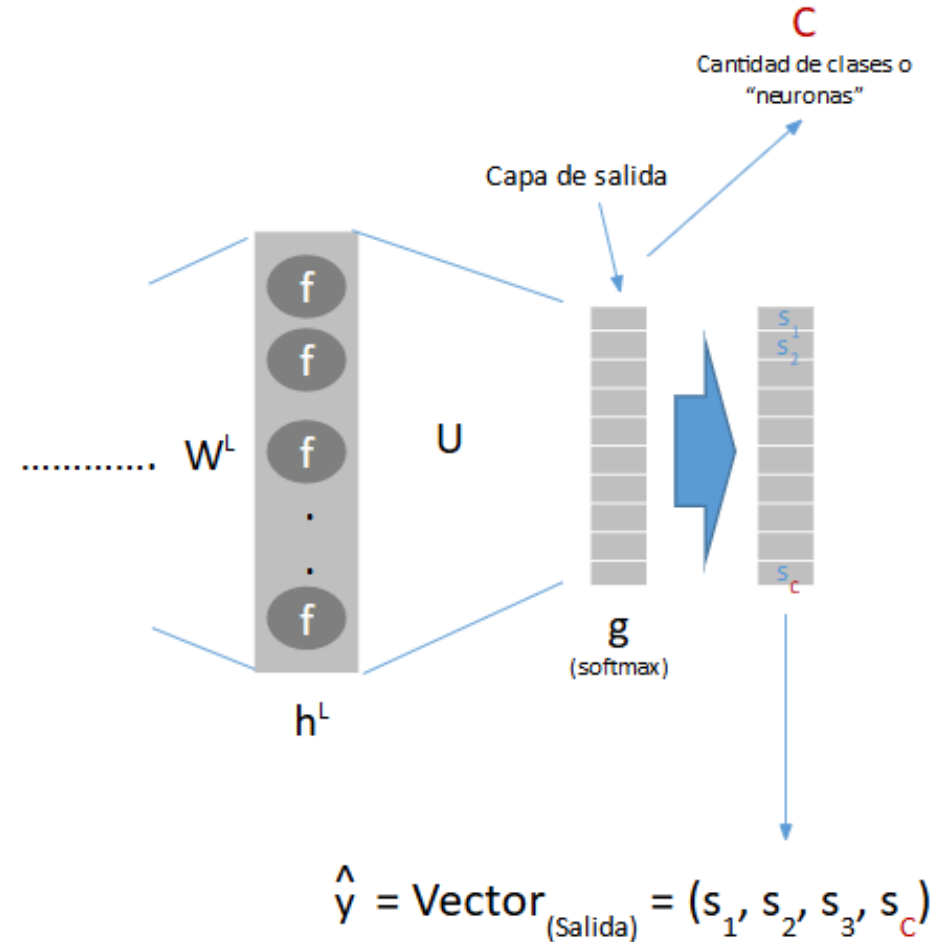
Función en la capa de salida

La función que se utiliza en la capa de salida era, o podría ser diferente, a las usadas como función de activación.

Esta función de salida va a depender del problema que quiero resolver (predicción o clasificación).

Para clasificación vamos a revisar una de las funciones más utilizadas: **Softmax**.

Cuando estamos clasificando, en general, interpretamos la salida como una distribución de probabilidades sobre las clases.



• Redes Feed Forward Fully Connected

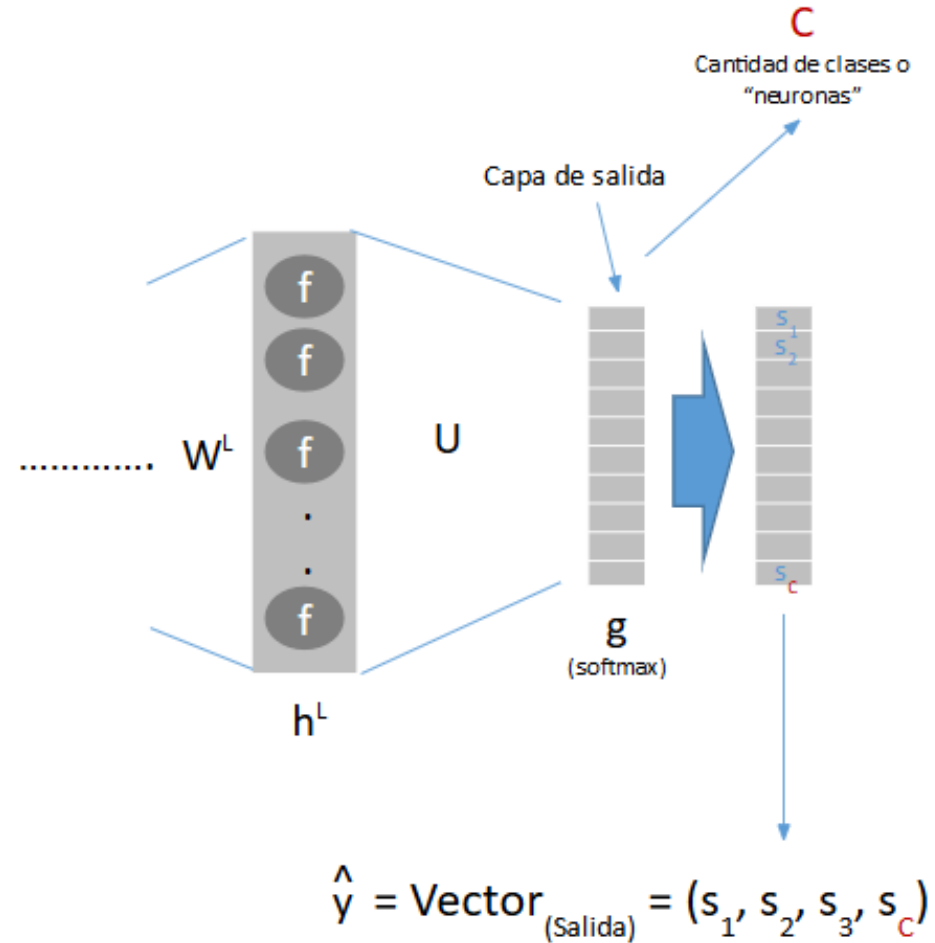
Función en la capa de salida

Definimos como clases a cuántas posibles salidas tenemos.

Por ejemplo:

Si estoy clasificando estados de ánimos: pena, alegría, depresión, ansiedad. Se dice que **tenemos cuatro clases**.

Por lo tanto, la salida de la red debería ser **un vector de cuatro neuronas**, donde cada una de ellas, representaría la probabilidad de que una de ellas sea la “respuesta o clasificación” correcta.



• Redes Feed Forward Fully Connected

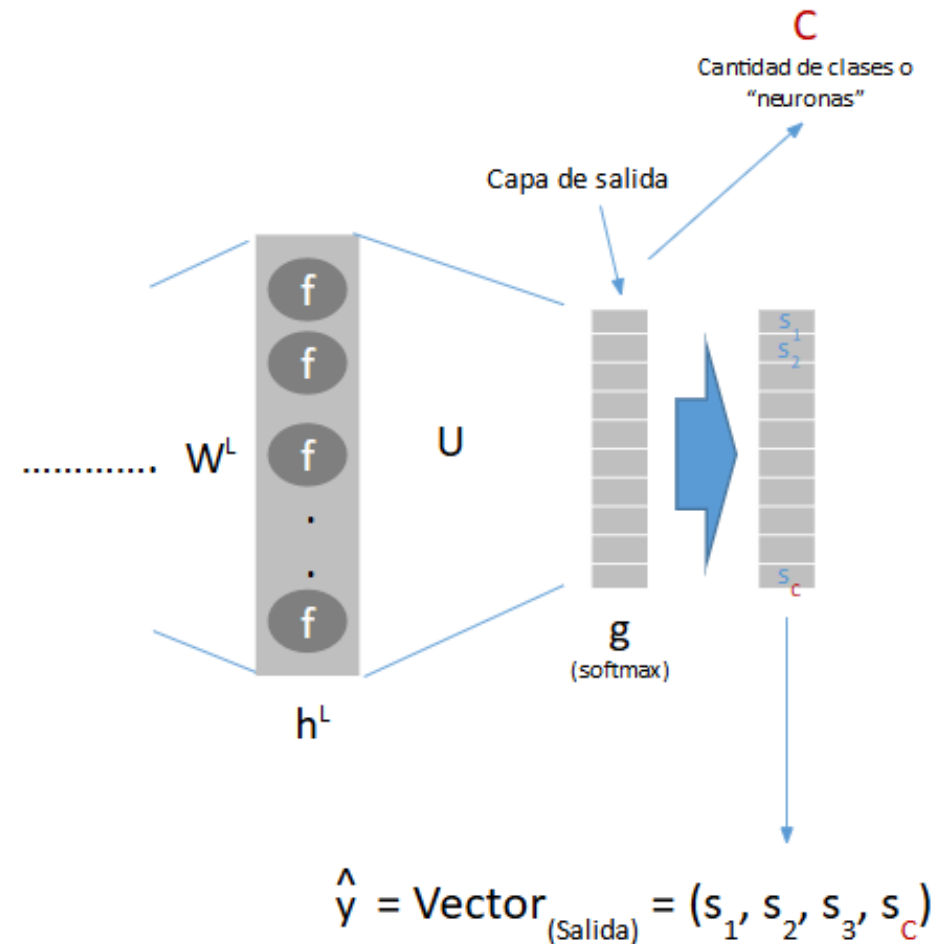
Función en la capa de salida

En otras palabras, si después de aplicar Softmax a la capa de salida, en una de las cuatro posiciones del vector de salida, hay un 0,90 en la clase “alegría”, significa que hay una probabilidad de un 90% que el estado de ánimo sea “alegría”.

Para lograr lo anterior, es necesario que la función de salida permita hacer dos cosas:

1. **Todos los números deben estar entre 0 y 1.**
2. **La suma de todos los números sea 1.**

Con esto, podría interpretar cada uno de los valores del vector de salida, como **una distribución de probabilidad**.



• Redes Feed Forward Fully Connected

Aspectos Técnicos

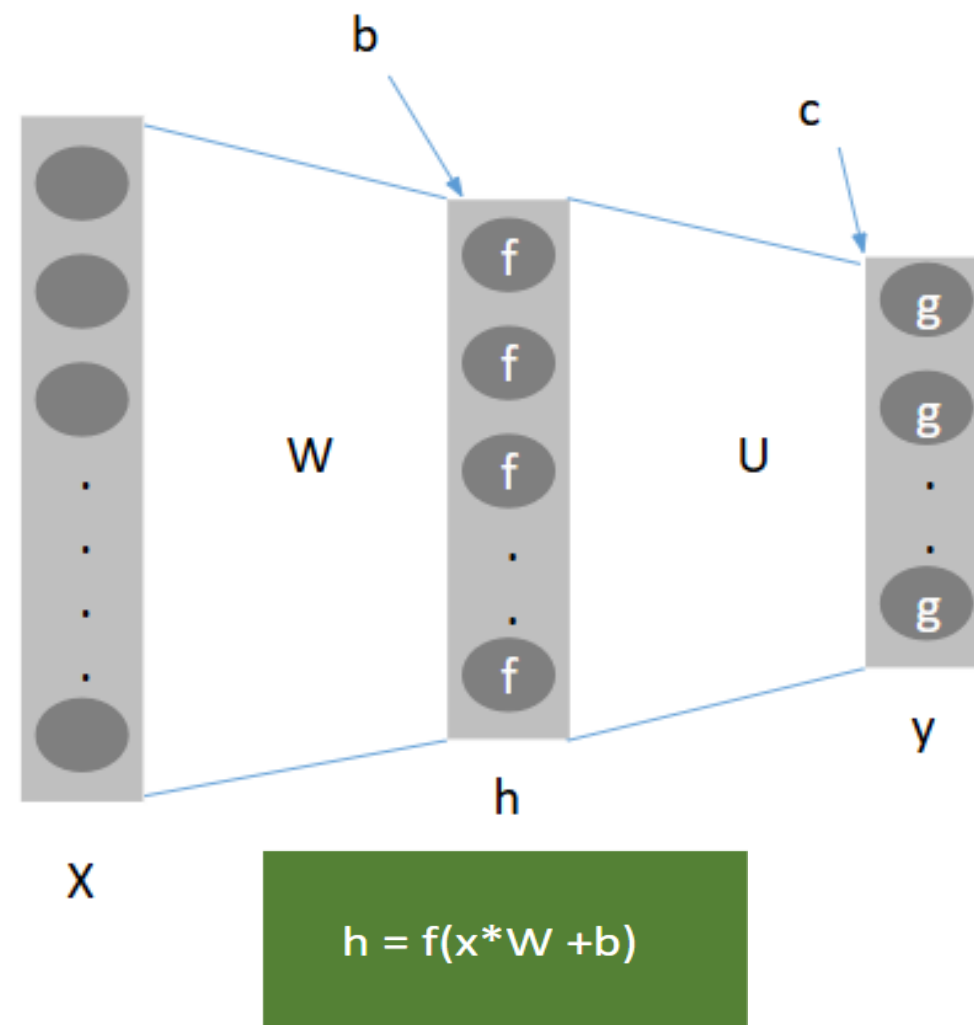
Matemáticamente, estas fórmulas **representan operaciones entre matrices**.

Multiplicar matrices es “caro” computacionalmente.

Una de las ventajas hoy, es usar una librería y hardware especializado (GPU, TPU)) que permita hacer las **operaciones con matrices de manera paralela**.

En la práctica, se pasan o ingresan a la red un conjunto de ejemplos, en lugar de pasar de un ejemplo a la vez de manera secuencial.

Entonces, X ya no sería un vector de $(k, 1)$, sino que sería una matriz de (k, n) .



A black and white photograph of a man and a woman in a modern office setting. The man, on the left, is wearing a light-colored button-down shirt and dark trousers. The woman, on the right, is wearing a dark blazer over a light-colored turtleneck. They are both looking at a tablet held by the man. In the background, there are office desks, computer monitors, and a person sitting at a desk. The overall atmosphere is professional and collaborative.

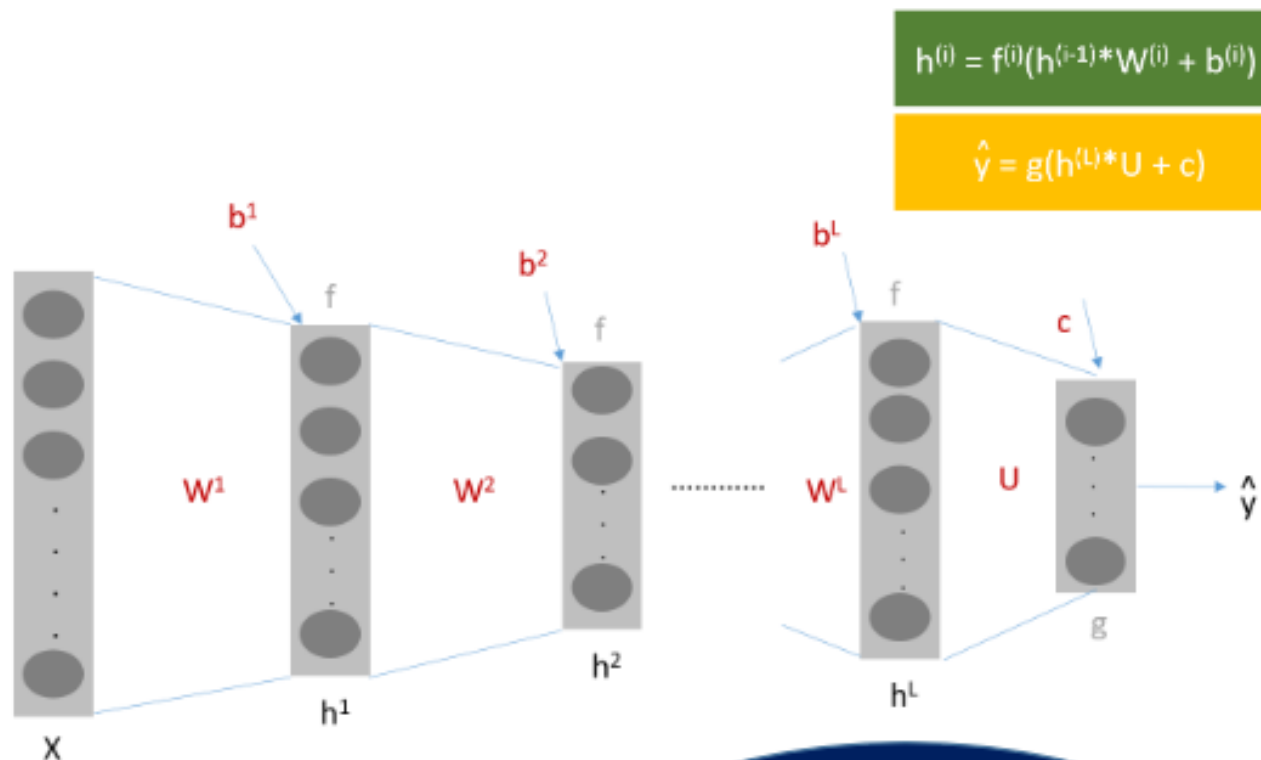
02

Entrenamiento

Entrenar significa que la red encuentre los valores correctos para todos los pesos y bias (denominados parámetros).

Para realizar el entrenamiento, se necesita calcular estos valores, los cuales son números reales, muy chicos (centrados en el cero).

La red itera, entre la primera capa oculta y la capa de salida hasta encontrar la mejor combinación de pesos y bias que resuelva el problema. Esto se conoce como **FORWARD**.

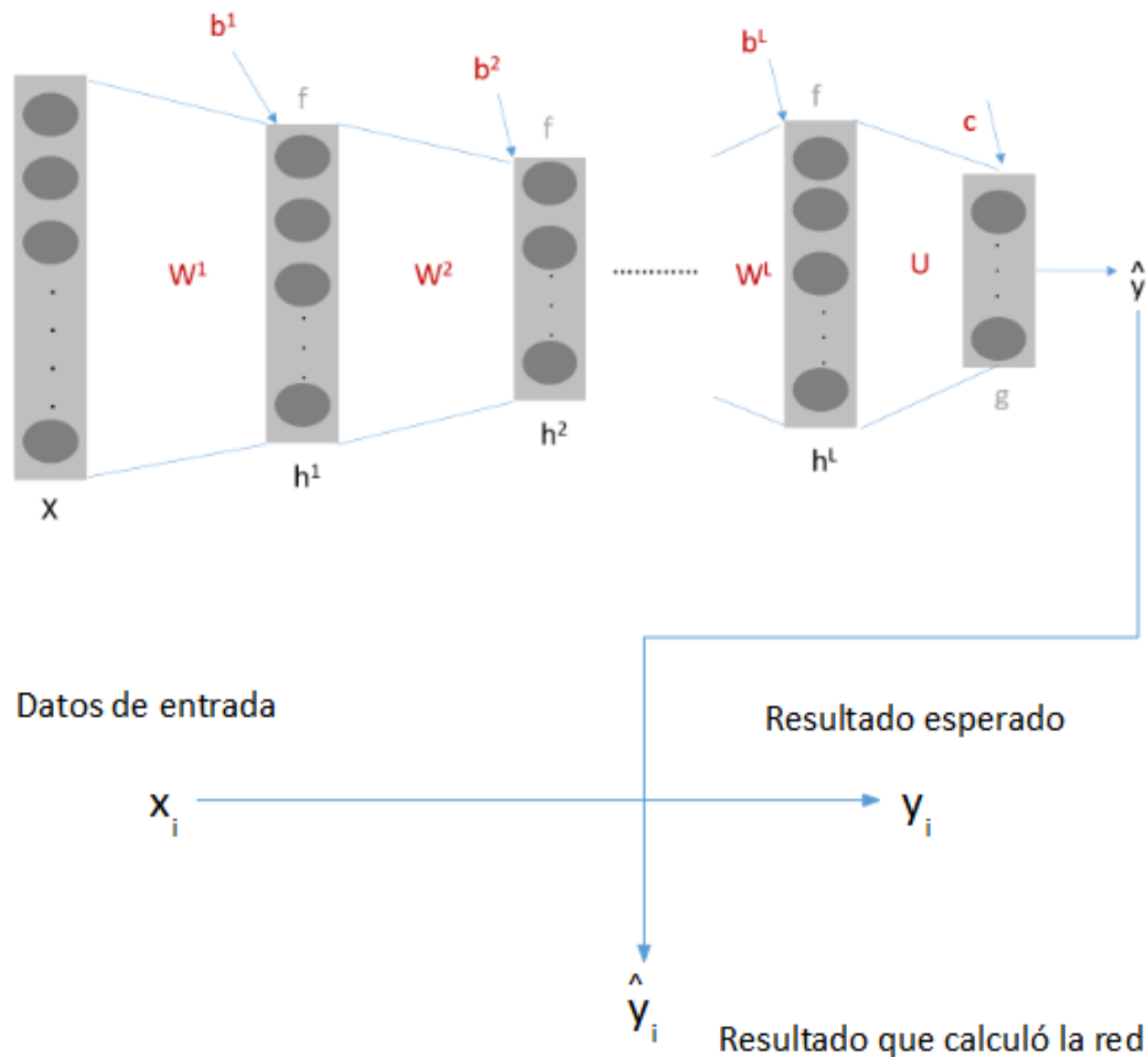


Entrenar una red neuronal es encontrar los valores "correctos" para W , b , U y c

Lo primero que se necesita para entrenar una red, son datos y en general necesitamos el dato y la salida (respuesta) que esperamos obtener.

Hasta el momento, hemos utilizado “ y ” para representar la salida calculada por la red. Sin embargo, “ y ” representa la etiqueta del dato ingresado, es decir, **es el valor a la cual la red debería llegar.**

Mientras que “ $\hat{y}$ ”, es el **resultado que calculó la red durante el entrenamiento.**



Para poder determinar si la red está calculando de manera correcta, necesitamos contar con alguna medida que permita **calcular la diferencia entre lo esperado y lo calculado**. Es decir, una medida que nos indique cuánto se está equivocando la red.

Este proceso se hace a través de una función. Esta función es denominada **función de error**.

La función de error (L) representaría la diferencia entre la salida esperada (y) y la salida calculada ($\hat{y}$).

$$L = (y_i ; \hat{y}_i)$$

Datos de entrada

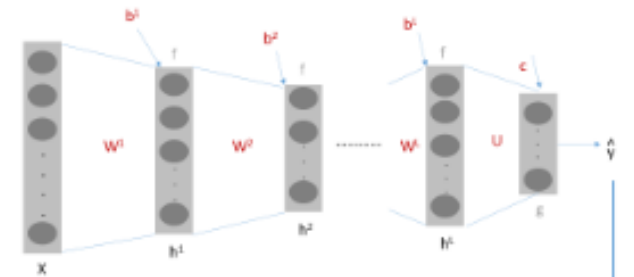
x_i

Resultado esperado

y_i

$\hat{y}_i$

Resultado que calculó la red



Es importante aclarar que el resultado de esta **función de error (L)** es un **número real**, un **escalar**, que sea lo más **cercano a cero**, de hecho, teóricamente igual a cero.

La fórmula nos indica el error o diferencia para un ejemplo.

Sin embargo, esto no es suficiente, lo que queremos es que la red se equivoque, en promedio, muy poco en todos los datos que hemos ingresados a la red.

$$L = (y_i ; \hat{y}_i)$$



$$L = \frac{1}{n} \sum_{i=1}^n \text{error}(\hat{y}^{(i)} ; y^{(i)})$$

Lo que hará la red entonces, es promediar todos los errores que comete por cada ejemplo.

A este error promedio de todos los errores cometidos en cada ejemplo, se le llama en Deep Learning “pérdida” o “loss”.

$$L = (y_i ; \hat{y}_i)$$



$$L = \frac{1}{n} \sum_{i=1}^n \text{error}(\hat{y}^{(i)} ; y^{(i)})$$

Es importante señalar y destacar, que esta función de error depende de los parámetros, de todos los parámetros de la red, esto lo representamos por un valor theta (θ) que es igual a todos los W_i , b_i , U y c .

Dado que definimos esta dependencia, si queremos que la función de error cambie e idealmente tienda a cero, debemos cambiar o ajustar los parámetros de la red.

Finalmente, podemos decir en términos matemáticos, que entrenar significa encontrar el valor de θ que minimiza la función de error (L).

$$L = \frac{1}{n} \sum_{i=1}^n \text{error}(\hat{y}^{(i)}; y^{(i)})$$



$$\theta = (W^i, b^i, U, c)$$

DuocUC[®]



CERCANÍA. LIDERAZGO. FUTURO.

7
AÑOS
ACREDITADO



NIVEL DE EXCELENCIA

HASTA OCTUBRE 2031

Docencia de pregrado / Gestión institucional / Aseguramiento interno de la calidad / Vinculación con el Medio / Investigación, creación y/o innovación